

PIMoE: Towards Efficient MoE Transformer Deployment on NPU-PIM System through Throttle-Aware Task Offloading

Lizhou Wu^{*†}, Haozhe Zhu^{*†}, Siqi He[†], Xuanda Lin[†], Xiaoyang Zeng[†], and Chixiao Chen^{†‡}

[†] State Key Laboratory of Integrated Chips and Systems (SKLICS), Fudan University, Shanghai, China

[‡] Shaoxin Laboratory, Shaoxing, Zhejiang, China

E-mail: zhuhz@fudan.edu.cn

Abstract—Mixture-of-experts (MoE) technique holds significant promise for scaling up Transformer models. However, the data transfer overhead and imbalanced workload hinder efficient deployment. This work presents PIMoE, a heterogeneous system combining processing-in-memory (PIM) and neural-processing-unit (NPU) to facilitate efficient MoE Transformer inference. We propose a throttle-aware task offloading method that addresses workload imbalance between NPU and PIM, achieving optimal task distribution. Furthermore, we design a near-memory-controller data condenser to address the mismatch of sparse data layout between NPU and PIM, enhancing data transfer efficiency. Experimental results demonstrate that PIMoE achieves $4.5\times$ speedup and $13.7\times$ greater energy efficiency compared to the A100, and $1.4\times$ speedup over a state-of-the-art MoE platform.

I. INTRODUCTION

Transformer-based large language models (LLMs) have performed remarkably across various natural language processing tasks. This exceptional performance is largely due to the unprecedented growth in model sizes over recent years [1], [2]. However, increasing model capacity results in higher computational costs and memory requirements, making training and deployment increasingly challenging.

The Mixture-of-Experts (MoE) technique has emerged as a promising approach to scaling model sizes while minimizing proportional increases in computational overhead [3]–[7]. In MoE-based Transformers, the sparse gating mechanism allows only a subset of experts to process an input token, resulting in lower computational expenses compared to non-MoE models with equivalent parameter counts. Consequently, MoE Transformer has been practically applied to LLM productions [2].

Although MoE Transformers significantly increase model capacity without greatly raising computation costs, deploying these models for inference faces two main challenges: data transfer overhead and workload imbalance. Firstly, the limited on-chip buffer capacity causes frequent eviction and reloading of expert parameters, leading to significant data transfer overhead. This issue is worse for experts that receive few tokens, resulting in low parameter reuse and memory-bound tasks that increase inefficiencies and latency. Secondly,

^{*} Lizhou Wu and Haozhe Zhu are co-first authors. Haozhe Zhu is the corresponding author.

This work was supported by the National Natural Science Foundation of China (NSFC) under Grants 62304047 and 62488101.

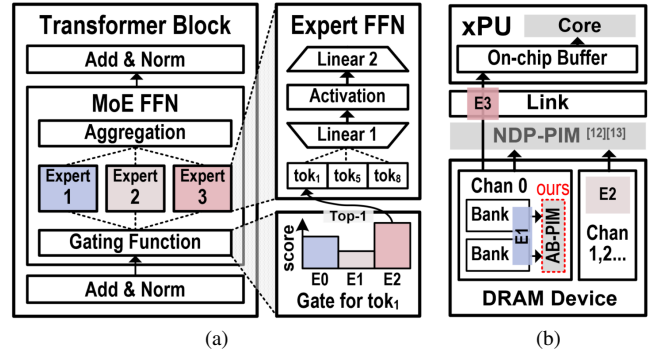


Fig. 1. (a) Diagram of Transformer block with MoE FFN layer. (b) Multi-levels of expert offloading in a heterogeneous NPU-PIM system.

the dynamic sparse allocation in MoE Transformers causes workload imbalance, with some experts overburdened while others are underutilized. This is particularly problematic in heterogeneous multi-core systems, where imbalanced token distribution leads to latency bottlenecks in the critical path.

Many works have integrated xPU (NPU/GPU) with PIM to address data transfer and memory overhead. IANUS [8], AttAcc [9], NeuPIMs [10] propose offloading memory-bound matrix-vector multiplications to PIM devices, leveraging the high bandwidth. Cambricon-LLM [11] proposes a NPU-NAND hybrid architecture to support tremendous storage demand. While designed for general transformer models, these works do not fully concern the specific expert imbalance in the MoE context. MoNDE [12] proposes a near-data-processing (NDP) approach for MoE Transformers. It trade-off parameter movement with activation movement based on the workload characteristics of different experts. Duplex [13] proposes a new HBM device with increased TSV amount to enlarge the NDP's bandwidth for better computing throughput.

In this work, we present PIMoE, a co-optimized NPU-PIM system for efficient MoE Transformer deployment. In summary, this paper makes the following contributions:

- We fully leverages the all-bank parallelism of commercial PIM products, addressing the memory-bounded expert computation associated with MoE FFN.
- We propose a throttle-aware task offloading method designed to optimize the distribution of experts be-

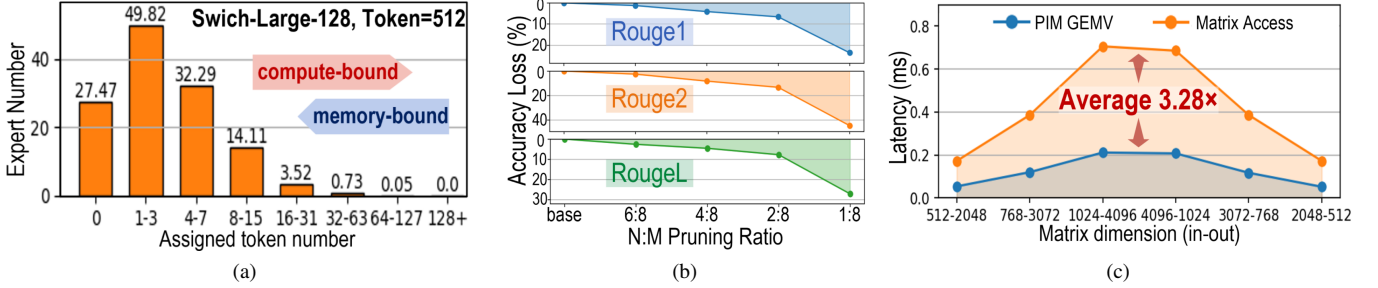


Fig. 2. Preliminary Experiments (a) Histogram of experts by allocated token count. (b) Accuracy degradation across various N:M pruning ratios. (c) Latency Comparison between PIM-based GEMV and matrix data loading.

tween PIM and NPU, mitigating performance bottlenecks caused by imbalanced workloads.

- We propose a near-memory-controller data condenser that addresses the mismatch between DRAM and NPU sparse data layouts. Condensing transfer data also alleviates NoC congestion and enhances data transfer efficiency.
- PIMoE achieves a $4.5\times$ speedup and $13.7\times$ energy efficiency compared to the A100. It also delivers a $1.4\times$ speedup over a state-of-the-art NDP MoE platform.

II. BACKGROUND AND MOTIVATION

A. MoE-based Transformers

The Mixture-of-Experts (MoE) technique [14] partitions a complex task into separated sub-tasks, enhancing the model's fitting ability. Sparse MoE [3] further introduces a gating network that selectively activates a subset of experts for each workload. As for Transformer models, the sparse MoE has been applied to the feed-forward layers (FFNs) [6], [7] to relax computation as the model increasingly scales up. Fig. 1 illustrates a typical MoE-based FFN (dubbed MoE FFN) within a Transformer block. The MoE FFN replicates the original FFN into N experts, each comprising two linear layers with an activation layer in between. At the block's entrance, the gating function infers the probability of each input token being allocated to the N experts. Then the token is routed to the top- K experts for inference. The results from each expert are weighted according to their corresponding probabilities and aggregated at the output.

The token workloads across FFN experts feature uneven distribution. We profile the switch-large-128 model [6] consisting of 128 experts and operates with 512 tokens. Fig. 2(a) shows the histogram of experts by allocated token count. Nearly half of the experts are allocated fewer tokens (i.e., 1-3), featuring memory-bound characteristics with limited weight reuse. In contrast, those experts with heavier token allocations are compute-bound, dominated by matrix multiplication.

Besides the sparse expert selection, the expert can further benefit from weight pruning, particularly the N:M method that selects N significant weights from each M -element vector. As indicated by Fig. 2(b), this approach reduces on-chip buffer overhead without severe degradation in performance. Furthermore, this weight sparsity can be effectively leveraged to accelerate computation by skipping the zero operation.

B. Commercial PIM Platforms

Processing-in-memory (PIM) covers a variety of techniques that deploy computing resources near or within DRAM devices. Among them, the all-bank (AB) paradigm, which deploys processing units besides the DRAM banks, has gained favor among several semiconductor manufacturers (PIM-HBM [15], AiM [16]). AB-PIM fully utilizes the internal bus bandwidth without intrusive change to the memory cell, striking a balance between throughput and storage density.

Commercial AB-PIM (e.g., PIM-HBM, AiM) prioritizes compatibility with existing protocols: High-level computing commands are converted into low-level JEDEC commands (write/read) to initiate PIM execution, typically managed by the memory controller. Therefore, both the PIM computation and the normal memory access occupy the memory controller, incrementally increasing the latency.

To characterize the PIM's performance, we profile the PIM-HBM with the corresponding PIMSimulator [17]. As shown in Fig. 2(c), the latency of per token matrix-vector multiplication (GEMV) linearly scales with the size of the weight parameters. Moreover, the throughput ratio between PIM computation and regular memory access is also nearly in proportion. Specifically, an N -to- M GEMV achieves $3.28\times$ higher throughput than loading an $N\times M$ weight parameter. This falls short of the ideal $8\times$ bank-level parallelism in PIM-HBM, due to PIM-specific overheads like activation writes and barrier insertions. This result indicates that PIM is well-suited for 'cold' experts with fewer tokens. For 'hot' experts with many tokens, the PIM's efficiency diminishes.

C. Motivation

To efficiently accommodate experts with diverse workloads, we propose an NPU-PIM hybrid system that leverages the PIM's weight-stationary feature and bank-level parallelism, as well as the NPU's high computational throughput. To prevent system performance from being throttled by critical experts on specific components, we propose a throttle-aware task offloading method that optimizes task allocation based on a latency model of the heterogeneous components. Furthermore, even though the NPU can effectively leverage sparsity, current PIMs do not well support sparse FFN inference, where the sparse weights have to be stored in a structured pattern. To bridge the gap of sparse data layout between NPU and PIM, and

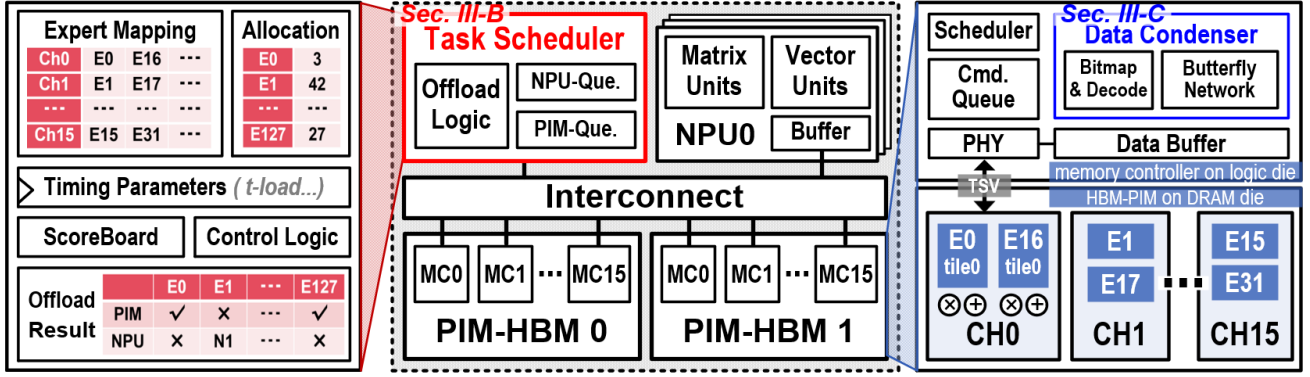


Fig. 3. The overall architecture of PIMoE. The task scheduler is explained in section III-B. The data condense is elaborated in section III-C.

enhance the bandwidth utilization for preventing congestion, we propose a near-memory-controller data condenser.

III. PIMoE

This section introduces the architecture of PIMoE, the throttle-aware task offloading method that optimizes the heterogeneous system, and the near-memory-controller data condenser that improves transfer efficiency.

A. PIMoE Architecture

Fig. 3 illustrates the overall architecture of the proposed PIMoE, the components of which are introduced below.

NPU: The NPU consists of matrix units (MUs), vector units (VUs), and scratchpad buffers. The MUs perform matrix multiplications involved in the gating function and the two linear layers within FFN experts. The MU is designed as a systolic array that supports sparse acceleration. The weights are pruned using the N:M scheme and are densely stored in the buffer, along with the corresponding bitmap. During computation, the activations pass through a gather unit that decodes the bitmap to select N non-zero elements from M inputs [18], while the multiply-accumulate (MAC) unit calculates the inner product of the condensed vectors. The VUs handle the element-wise operations including bias addition, residual addition, activation function, and weighted sum of multiple experts' results.

PIM: PIMoE adopts Sumsang's commercial product, PIM-HBM [15], which is based on HBM2 DRAM that consists of 16 channels, offering a total of 1024 I/O pins and a maximum bandwidth of 307GB/s. Each channel contains 16 banks, with PIM-HBM deploying 8 processing units among these banks. Due to the limited routing resources in DRAM devices, it is preferable to localize the computation with the channel to minimize the global communication and synchronization overhead. Therefore, each expert's parameters are consecutively mapped within the channel's address range and only tiled across different DRAM devices to fit the per-channel storage limit. To support straightforward scaling of the number of experts, the experts are interleaved across channels, with the E_{th} expert being mapped to the channel $E_{th} \% N_{chan}$. Each DRAM channel includes a memory controller that supports both normal memory access and PIM-specific commands. The memory controller is equipped with a data condenser,

as discussed in Section III-C, to efficiently transfer structured weight parameters sparsely stored in DRAM to the sparsity-optimized NPU that requires a dense weight format.

Task Scheduler: The task scheduler dispatches the experts across the NPUs and PIMs based on the runtime token distribution. It takes the results of the gating function as input and decides each expert's computing medium according to the throttle-aware offloading method described in Section III-B. For the NPU-offloaded expert, it requests the DMA to load the parameters from DRAM to the on-chip buffer and triggers the matrix multiplication once the data is prepared. For the PIM-offloaded task, it issues the PIM commands to the target memory controller according to the expert address and loop for each assigned token. To avoid inefficiency caused by frequent mode switching, the task scheduler inserts barriers between two types of tasks.

Network-on-chip: The NoC manages data transfer between the DRAM and NPU. It employs a 2D mesh topology with NPUs and memory controllers as nodes. For PIM-offloaded tasks, the NPU initiates the transfer, sending activation data to the target DRAM partition. For NPU-offloaded tasks, expert parameters are loaded from DRAM to the target NPU.

B. Throttle-Aware Task Offloading Method

The heterogeneous platform introduces more factors regarding workload imbalance, which can be categorized as PIM-throttle and NPU-throttle within the context of PIMoE.

PIM throttle: For a batch of input tokens offloaded to PIM, each DRAM channel handles those allocated to the subset of experts within its address range. The memory controller operates in parallel, with the critical channel managing the largest number of tokens. Due to the requirement for output synchronization across experts, the critical channel constrained the execution latency of the PIM-offloaded MoE FFN.

NPU throttle: For those experts offloaded to NPU, their parameters are loaded from the DRAM to the on-chip buffer. As the number of loaded experts increases, the latency of the critical NPU may exceed that of the PIM latency, causing the latter to wait for the completion of the NPU.

Existing work, such as IANUS [8] and MoNDE [12], employs a straightforward offloading method, not fully accounting for potential runtime throttle. IANUS offloads a task

Algorithm 1: throttle-aware offloading method

Input: M_{CH-EP} , M_{EP-TK} , $threshold$

Output: M_{EP-PIM}

```

1  $M_{EP-PIM} = [EP]$ ;
2 Initialize( $[t_{CH}]$ ,  $[t_{NPU}]$ ,  $M_{EP-TK}$ );
  // Until NPU becomes the throttle
3 while  $\max([t_{CH}]) < \max([t_{NPU}])$  do
4    $CH_{thro} = \text{argmax}([t_{CH}])$ ;
5    $EP_{candi} = M_{CH-EP}[CH_{thro}] \cap M_{EP-PIM}$ ;
6   if  $EP_{candi} \neq \emptyset$  then
7      $N_{EP-TK} = M_{EP-TK}[EP_{candi}]$ ;
8      $EP_{thro} = \text{argmax}(N_{EP-TK})$ ;
9      $t_{PIM} = \text{GetPimLatency}(EP_{thro})$ ;
10    if  $N_{EP-TK}[EP_{thro}] > threshold$  then
11      // Offload the expert to NPU
12       $M_{EP-PIM} = M_{EP-PIM} \setminus EP_{thro}$ ;
13       $NPU_{target} = \text{argmin}([t_{NPU}])$ ;
14       $t_{NPU}[NPU_{target}] += t_{LD}$ ;
15       $t_{CH}[CH_{thro}] += t_{LD} - t_{PIM}$ ;
16    else
17      break; // The CH has optimized
18    end
19  else
20    break; // No experts left for PIM
21 end

```

to PIM if its latency is lower than that of on-chip execution. MoNDE adjusts the number of experts to offload to NDP based on the bandwidth ratio between the NDP and PCIe. We thus propose a throttle-aware offloading method to address performance limitations caused by PIM and NPU throttle.

Given that the memory controller processes normal access and PIM commands sequentially, each DRAM channel latency (t_{CH}) includes two components: the expert loading latency (t_{LD}) and the PIM execution latency (t_{PIM}). The latency t_{PIM} for a given expert is the product of the number of offloaded tokens N_{TK} and the latency for one PIM-based FFN, t_{FFN} , which itself includes matrix-vector multiplication latency t_{GEMV} and element-wise operation latency t_{ELEM} . The loading latency t_{LD} is determined by the parameter size S_{EP} and the channel's effective bandwidth BW . As for the NPU latency, it is approximated by the product of the number of offloaded experts N_{EP} and the loading latency t_{LD} , considering the tiled pipeline execution and data transfer bottleneck. The timing model is given below:

$$t_{CH} = \sum t_{LD} + \sum t_{PIM} \begin{cases} t_{LD} = S_{EP} \div BW \\ t_{PIM} = N_{TK} \times t_{FFN} \end{cases} \quad (1)$$

$$t_{NPU} = N_{EP} \times (t_{LD} + t_{EXE}) \approx N_{EP} \times t_{LD} \quad (2)$$

Algorithm 1 depicts the process of the proposed offloading algorithm. The expert-channel mapping table (M_{CH-EP}) and expert-token allocation table (M_{EP-TK}) are taken as inputs to

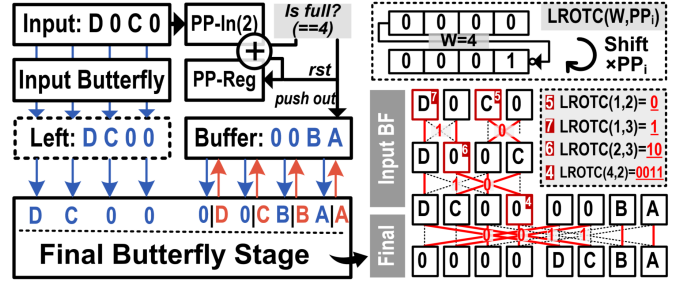


Fig. 4. Diagram of the near-memory-controller data condenser.

generate the PIM-expert table (M_{EP-PIM}), which represents the set of experts offloaded to PIM. Initially, all experts are offloaded to the PIM. The algorithm iteratively selects a target expert to offload to the NPU until the NPU becomes the bottleneck. It begins by identifying the channel with the highest latency and checking for any active experts in that channel. If there are experts whose workloads exceed the defined *threshold* (the latency ratio of matrix loading to PIM-GEMV, as noted in Section II-B), the expert with the most tokens is selected. It is then offloaded to the NPU with the least latency, and the latency scoreboards for both the NPU and PIM are updated according to the predefined model.

At the circuit level, the offloading algorithm is hardcoded. The timing parameters (i.e., $t_{LD}/t_{GEMV}/threshold$) are implemented as registers, which can be programmed with measured values. The offloading logic looks up the parameters and places the task into the target queue for dynamic scheduling.

C. Near-memory-controller Data Condenser

Given DRAM's limited logic budget, PIM struggles with irregular, non-extreme sparse GEMV, whereas NPUs can reorganize sparse computation into a densified pattern. Additionally, DRAM transfers are triggered in bursts, accessing consecutive data that may contain zero elements. This brings a mismatch in data layout between PIM and NPU, and leads to underutilization of NoC bandwidth and potential congestion.

To address this, we deploy a near-memory-controller data condenser to filter out zero elements from the input buffer, injecting dense 256-bit flits into the NoC. The condenser's core is a butterfly network reorganizing scattered non-zero elements into a consecutive sequence. As shown in Fig. 4, the butterfly network consists of multiple switch layers, each controlled by a bit to determine whether to swap the inputs. Using the input vector's bitmap and the rightmost offset, the LROT (left rotate and complement on wrap) operation [19] recursively generates configuration bits for each switch layer, implemented through nested bitwise barrel rotators. A counter tracks the number of non-zero elements in the output buffer to align the inputs and outputs. This counter provides the offset to guide the LROT within the input stage of the butterfly network, which permutes the input vector. The final butterfly stage further consolidates the reordered input into the output buffer. After each input cycle, the counter is incremented by the input's pop-count (i.e., the number of non-zero elements). Once the counter reaches its threshold, indicating that the

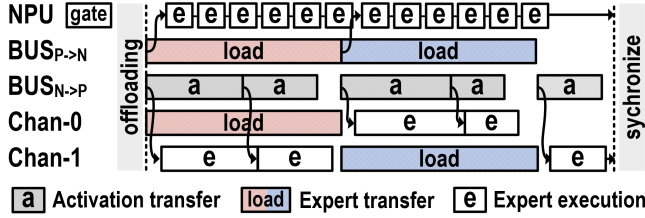


Fig. 5. Behavior model of the PIMoE simulator.

buffer is fully populated with dense elements, the data is pushed to the output queue, and the counter resets.

IV. EVALUATION

A. Experimental Setup

Methodology: To evaluate the performance of the PIMoE platform, we develop a simulator composed of several sub-systems. For the PIM subsystem, we use Samsung’s open-source PIM-HBM simulator, PIMSimulator [17], which is based on the cycle-accurate DRAMSim [20]. PIMSimulator implements a PIM-specific command set to support common PIM testbenches, such as GEMV and element-wise computations. Regarding the execution latency of a PIM-offloaded expert, we simulate the two linear layers and an intermediate activation layer. With barriers inserted between each phase, the total latency is computed as the sum of the individual phases, which are executed sequentially. For the NPU subsystem, a pipelined scheme is adopted for weight loading and computation, where the expert is tiled for fine-grained execution. Therefore, the data transfer determines the latency, which is also evaluated by the PIMSimulator. For the NoC subsystem, we utilize the noxim simulator [21] to evaluate the congestion. We generate synthetic traffic patterns to represent the practical transfer workload in PIMoE. The injection rate is determined by the ratio of the DRAM channel’s I/O throughput to the product of flit size (i.e., the NoC bus width, set as 256 bits) and the packet size (set as 8). The behavior model

TABLE I
PIMoE SYSTEM CONFIGURATION AND WORKLOAD DETAILS

PIMoE	Composition	4 NPU, 2 PIM-HBM		
NPU	Frequency	1GHz		
	Matrix Unit	4096 MAC units, 8.2 TFLOPS		
	Scratchpad	Activation 3MB, Weight 1MB		
PIM	Memory configuration	HBM2, 2GB(4H), 2.4Gb/s, 307GB/s 16 channels/cube, 16 banks/channel		
	Timing parameters	tCK=1ns, tCCDS=2ns, tCCDL=4ns, tRAS=33ns, tWR=16ns, tRP=14ns, tRCDRD=14ns, tRCDWR=10ns		
	Processing Unit (PU)	300MHz, 8 PUs per channel 9.6 GFLOPS per PU		

Model	d_{model}	d_{ff}	N_{expert}	Expert Params
Switch-Large-128	1024	4096	128	48 GB
Switch-Base-16	768	3072	16	1.7 GB

of PIMoE for simulating MoE FFN execution is illustrated in Fig. 5. Initially, the simulator determines the offloading based on the proposed throttle-aware offloading method. Then it tracks the state of each processing unit based on the pre-evaluated latencies. Once the occupancy of a unit is released, the simulator schedules a new task for execution.

Implementation: Table I presents the configuration of the PIMoE system. To accommodate an entire MoE FFN layer on PIMoE, we integrate two PIM-HBM devices with a total capacity of 4GB. This capacity is sufficient to store the layer-wise weight parameters (2GB per switch-large-128 MoE FFN layer), intermediate results, bitmaps, and other necessary data. Each expert is tiled along the output channels, enabling concurrent execution across the two devices. For GPU comparison, we measure the latency of the MoE layers using PyTorch’s CUDA Event API, placing hooks at the entry and exit points of the inference process. To evaluate the energy consumption, we refer to the physical parameters provided in [23]. To evaluate the area overhead of the data condenser on the logic die, we implement it with RTL using a commercial 28nm technology library and synthesize it with Design Compiler.

Workloads: We evaluate PIMoE using the pre-trained Switch Transformers in the Hugging Face repository. We adopt the bfloat16 datatype, which is widely adopted for inference tasks. Since PIMoE is primarily customized for MoE tasks, we conduct our experiments solely on the MoE FFN layers. The details of the model are listed in Table I. As for the datasets, we choose the C4 dataset, a large-scale web-crawled text collection designed for evaluating language models. When evaluating performance under different token numbers, we disable padding and truncate sequences that exceed the target token count. This ensures that only effective tokens are considered during execution.

B. Experimental Analysis

Comparison of Performance. Fig. 6 compares the execution latency of an MoE FFN layer. Each scenario is normalized to the GPU baseline, NVIDIA’s A100. For the switch-base-16 model, PIMoE achieves an average of $1.6\times$ speedup compared to the A100. For the larger switch-large-128 model, PIMoE demonstrates a significant latency reduction of $8.47 \sim 3.25\times$ across various token numbers. The results indicate that, with fewer total tokens, the memory-bound cold experts become more prominent, allowing PIM execution to yield greater

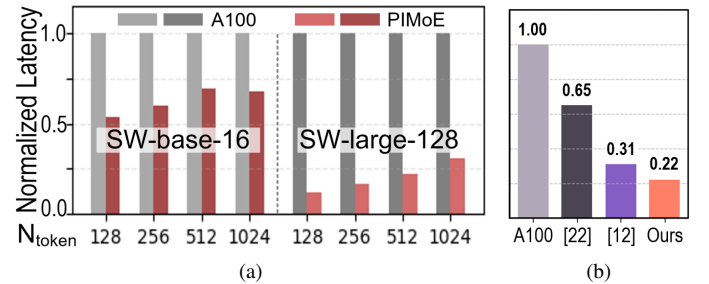


Fig. 6. Comparison of normalized latency (a) between A100 and PIMoE (b) between FLAME [22], MoNDE [12], and PIMoE.

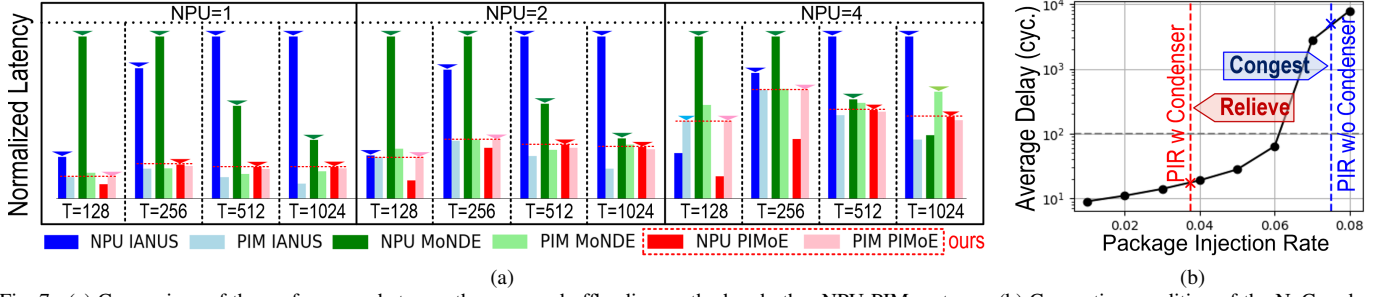


Fig. 7. (a) Comparison of the performance between the proposed offloading method and other NPU-PIM systems. (b) Congestion condition of the NoC under different packet injection rates (PIR).

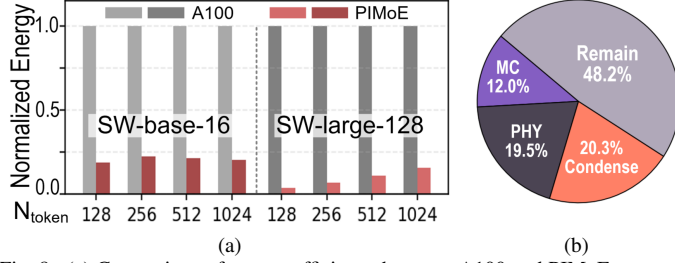


Fig. 8. (a) Comparison of energy efficiency between A100 and PIMoE across different scenarios. (b) Area breakdown of the logic die.

performance gains. We also compared PIMoE with other state-of-the-art accelerators for MoE transformers, specifically FLAME [22] and MoNDE [12]. We use their reported results normalized to the same GPU baseline (i.e., A100). Results show that PIMoE achieves an average improvement of $1.4\times$ over MoNDE. PIMoE benefits from the higher bandwidth of bank-wise parallelism, unlike the channel-wise parallelism adopted in MoNDE. Additionally, PIMoE’s tighter coupling between NPU and PIM bypasses the PCIe bottleneck in the GPU and CXL-based DRAM.

Comparison with other offloading scheme. Fig. 7(a) compares the effect of the proposed throttle-aware offloading scheme with the load balancing methods introduced in IANUS [8] and MoNDE [12], respectively. We implement their schemes on the PIMoE hardware baseline to ensure an apples-to-apples comparison. Each scheme’s NPU and PIM latencies are shown separately to intuitively highlight the system’s throttle points, indicated by a triangular arrow at the top of the corresponding bar. We also evaluate the sensitivity to varying numbers of NPUs and token workloads. For each scenario, the latencies are normalized to the highest value for comparison. The results show that throttle-aware offloading maintains a balance between NPU and PIM, particularly under heavy total token workloads and fewer NPU resources, leading to optimal performance across various configurations.

Effect of the data condenser. The number of effective packets injected into the NoC can be reduced through near-memory-controller data condensing. Particularly, the injection rate is halved with a 4:8 pruning. This prevents the NoC congestion under concurrent expert weight loading. Fig. 7 shows the average delay of data transfer under different packet injection rates (PIR). When no condensing is applied, the injection

rate reaches 0.075, resulting in significant delays (over 5000 cycles) and indicating severe congestion. In contrast, with the condenser, NoC saturation is postponed. At an injection rate of 0.0375, the average delay is reduced to around 16 cycles, resulting in a negligible impact on overall performance.

Area and Power Consumption. As shown in Fig. 8, PIMoE achieves an average $4.8\times$ and $13.7\times$ improvement in energy efficiency compared to the A100 for the switch-base-16 and switch-large-128 models, respectively. This energy saving is attributed to the reduced external data transfer energy enabled by near-bank PIM. As for the near-memory-controller data condenser, we compare its area requirements with those of the memory controller, PHY, and the available area on the logic die. By efficiently utilizing the area budget, the condenser alleviates NoC congestion with a modest 20.3% overhead.

C. Discussion on Scalability

As the expert amount scales up, additional PIM-HBM devices can be added, with each expert tiled across the devices. While this increases NoC complexity, the data condensers mitigate the NoC’s transfer burden, preventing congestion in larger configurations. Additionally, the proposed throttle-aware offloading method analyzes and schedules the complex expert distribution, optimizing resource usage and minimizing bottlenecks. This enables efficient handling of larger models and expert configurations while maintaining performance. By programming the timing parameters, this method can be applied to different NPU-PIM platforms.

V. CONCLUSION

In this paper, we presented PIMoE for efficient MoE Transformer deployment. By leveraging all-bank parallelism, PIMoE effectively addresses memory-bound challenges in MoE FFN computations. The throttle-aware task offloading mitigates workload imbalances between PIM and NPU. Additionally, the near-memory-controller data condenser improves data transfer efficiency by compressing sparse weight parameters from DRAM. Our experimental results show that PIMoE achieves $4.5\times$ speedup and $13.7\times$ greater energy efficiency compared to the A100, and $1.4\times$ speedup over a state-of-the-art MoE platform. These results demonstrate the potential of PIMoE for improving performance and energy efficiency in large-scale MoE Transformer systems, providing a solid foundation for future developments in this area.

REFERENCES

- [1] T. B. Brown, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *arXiv preprint arXiv:1701.06538*, 2017.
- [4] M. Artetxe, S. Bhosale, N. Goyal, T. Mihaylov, M. Ott, S. Shleifer, X. V. Lin, J. Du, S. Iyer, R. Pasunuru *et al.*, “Efficient large scale language modeling with mixtures of experts,” *arXiv preprint arXiv:2112.10684*, 2021.
- [5] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, “DeepSpeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale,” in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [6] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [7] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat *et al.*, “Glam: Efficient scaling of language models with mixture-of-experts,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 5547–5569.
- [8] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin *et al.*, “Ianus: Integrated accelerator based on npu-pim unified memory system,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 545–560.
- [9] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “Attacc! unleashing the power of pim for batched transformer-based generative model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 103–119. [Online]. Available: <https://doi.org/10.1145/3620665.3640422>
- [10] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, “Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 722–737. [Online]. Available: <https://doi.org/10.1145/3620666.3651380>
- [11] Z. Yu, S. Liang, T. Ma, Y. Cai, Z. Nan, D. Huang, X. Song, Y. Hao, J. Zhang, T. Zhi *et al.*, “Cambricon-llm: A chiplet-based hybrid architecture for on-device inference of 70b llm,” *arXiv preprint arXiv:2409.15654*, 2024.
- [12] T. Kim, K. Choi, Y. Cho, J. Cho, H.-J. Lee, and J. Sim, “Monde: Mixture of near-data experts for large-scale sparse models,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC ’24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3655951>
- [13] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, “Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching,” *arXiv preprint arXiv:2409.01141*, 2024.
- [14] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [15] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, O. Seongil, H.-S. Yu, H. Lee, S. Y. Kim *et al.*, “25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2 tflops programmable computing unit using bank-level parallelism, for machine learning applications,” in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 350–352.
- [16] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim *et al.*, “A 1ynm 1.25 v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [17] SAIT, “Pimsimulator,” <https://github.com/SAITPublic/PIMSimulator>, 2024.
- [18] C. Fang, A. Zhou, and Z. Wang, “An algorithm–hardware co-optimized framework for accelerating n: M sparse transformers,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 11, pp. 1573–1586, 2022.
- [19] Y. Hilewitz, Z. J. Shi, and R. B. Lee, “Comparing fast implementations of bit permutation instructions,” in *Conference Record of the Thirty-Eighth Asilomar Conference on Signals, Systems and Computers*, 2004., vol. 2. IEEE, 2004, pp. 1856–1863.
- [20] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [21] V. Catania, A. Mineo, S. Monteleone, M. Palesi, and D. Patti, “Noxim: An open, extensible and cycle-accurate network on chip simulator,” in *2015 IEEE 26th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2015, pp. 162–163.
- [22] X. Lin, H. Tian, W. Xue, L. Ma, J. Cao, M. Zhang, J. Yu, and K. Wang, “Flame: Fully leveraging moe sparsity for transformer on fpga,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC ’24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3656507>
- [23] S. Kim, S. Kim, K. Cho, T. Shin, H. Park, D. Lho, S. Park, K. Son, G. Park, S. Jeong *et al.*, “Signal integrity and computing performance analysis of a processing-in-memory of high bandwidth memory (pim-hbm) scheme,” *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 11, no. 11, pp. 1955–1970, 2021.